

AI-Powered Farmer Advisory System — Project Brief

(Multi-Domain Major Project — Security-Integrated)

1. Problem Statement

Small-scale Indian farmers face compounding challenges: crop diseases are often caught too late, crop and land decisions are made without reliable data support, and many farmers remain unaware of government schemes they are eligible for. Existing solutions (including recent academic systems like SCAS) address diagnosis and recommendation reasonably well, but they largely assume the user is literate, has stable internet access, and only needs an answer — not a usability-tested, actionable, or personally relevant one.

We are building a voice-first, explainable, low-connectivity-friendly farmer advisory platform that not only diagnoses crop disease and recommends crops, but also matches farmers to government schemes they actually qualify for — validated through real usability testing, and hardened as a secure, production-realistic system rather than a lab prototype.

2. What We're Building

1. **Crop Disease Detection** — image-based diagnosis using a trained CNN (MobileNetV2/EfficientNet), tested on both clean and real-world (messy) field images.
 2. **Explainable Diagnosis** — Grad-CAM heatmap overlay on the leaf photo showing where disease was detected, not just a text label.
 3. **Crop Recommendation** — soil, season, weather, and market-based recommendation using an explainable model (e.g. decision tree).
 4. **Government Scheme Matching** — rule-based eligibility engine matching farmer profile (land size, crop, state, situation) to real schemes (PMFBY, PM-KISAN, Soil Health Card, KCC).
 5. **Voice-First Interface** — speech input/output in 2–3 languages (Hindi, English + one regional language) for low-literacy accessibility.
 6. **Offline-First Design** — core features usable with poor/no connectivity (PWA with caching/sync).
 7. **Confidence-Aware UI** — the system flags low-confidence predictions and suggests next steps instead of forcing a guess.
 8. **Real Usability Study** — task success rate, comprehension checks, SUS/NASA-TLX with actual test users, not just expert review.
 9. **End-to-End Security Layer** — LLM/model integrity, API hardening, data privacy, and offline-cache security built in as a core system property, not an afterthought.
-

3. Novelty (vs. Existing Systems like SCAS)

Gap in Existing Systems	Our Approach
Text-only LLM explanations	Visual (Grad-CAM) + voice explanation
No uncertainty handling	Confidence-aware UI with fallback guidance
Online-only	Offline-first PWA
Language support = translated text only	True voice-first design, not just multilingual text
No government scheme support	Eligibility-matched scheme recommendations
Expert-rated evaluation only	Real farmer-facing usability study
Diagnosis → advice	Diagnosis → recommendation → actionable entitlement
No security consideration for AI/ML pipeline or sensitive farmer data	Adversarial robustness testing, model integrity checks, prompt injection defense, encrypted PII, offline cache security

4. Tech Stack

- **ML:** Python, PyTorch/TensorFlow, MobileNetV2/EfficientNet, Grad-CAM, scikit-learn (crop recommendation)
- **LLM layer:** Anthropic/Gemini API (explanation generation only, not decision-making)
- **Voice:** Google Cloud Speech-to-Text / Web Speech API, Google Cloud TTS or gTTS
- **Backend:** NestJS + TypeScript (orchestration, auth, scheme data), FastAPI (Python ML inference microservice)
- **Database:** PostgreSQL
- **Frontend:** React + Vite, PWA (vite-plugin-pwa) for offline support
- **Deployment:** Vercel/Netlify (frontend), Render/Railway (backend + ML service)
- **Usability tooling:** Figma (prototyping), SUS/NASA-TLX questionnaires, OBS (session recording)
- **Security tooling:** JWT auth, rate limiting middleware, npm audit / pip-audit (dependency scanning), TLS, column-level encryption (Postgres), HMAC for cache integrity, adversarial testing scripts (e.g. via `foolbox`/`cleverhans`) or custom perturbation scripts)

5. Team Division (4 Members)

Person A — ML & Explainability

- Crop disease detection (CNN: MobileNetV2/EfficientNet), trained and tested on clean and messy field images
- Grad-CAM heatmap overlay for explainable diagnosis
- Crop recommendation model (decision tree — soil/season/weather/market based)
- Confidence-calibrated output (model returns usable confidence scores, not just a label — consumed downstream by both the UI and the security adversarial-testing work)

Person B — Backend & Data

- NestJS orchestration layer + PostgreSQL schema (farmer profiles, diagnoses, scheme matches)
- FastAPI ML inference microservice + API contracts between the two
- Government scheme rule engine (PMFBY, PM-KISAN, Soil Health Card, KCC eligibility logic)
- LLM integration for explanation generation (Anthropic/Gemini API call layer)

Person C — Frontend, Voice & Offline UX

- React + Vite frontend, PWA setup (vite-plugin-pwa)
- Voice input/output in Hindi, English + one regional language
- Offline-first sync/caching logic
- Owns the usability study: task success rate, SUS/NASA-TLX, session recording, comprehension checks

Person D — Security (cross-cutting, all layers)

Not a bolt-on: security reviewer and implementer across A, B, and C's components, plus original contributions.

- **LLM security:** Prompt injection defense on the explanation layer (input sanitization, output validation so explanations can't be steered off the actual diagnosis data)
 - **Model integrity & adversarial robustness:** Checksum/signing pipeline for trained model weights; adversarial robustness testing (perturbed leaf images vs. diagnosis stability) — the most novel, independently write-up-able contribution
 - **API/backend security:** JWT auth + role separation, rate limiting on NestJS/FastAPI services, input validation on all upload endpoints
 - **Data privacy:** Encryption at rest for sensitive farmer PII (land size, location, scheme eligibility) in Postgres, TLS in transit, audit logging on scheme-matching decisions
 - **Offline/PWA security:** Encrypted local cache for offline-stored data, sync integrity checks (HMAC on cached records) on reconnect
 - **Supply chain security:** Dependency scanning (npm audit / pip-audit) wired into CI
-

6. Additional Domains Worth Folding In (Optional Extensions)

- **Digital identity:** Aadhaar-based (or mocked) identity verification for scheme eligibility — reduces fraud risk in the scheme-matching engine.
 - **Agri-fintech:** KCC (Kisan Credit Card) application flow integration — closes the loop from "eligible for scheme" to actual entitlement, reinforcing the novelty table's diagnosis → recommendation → actionable entitlement claim.
 - **HCI/accessibility research:** The usability study is already a formal domain in its own right — worth citing explicitly alongside ML and Security in report framing, since usability-validation (not just accuracy) is a core differentiator from systems like SCAS.
-

7. Evaluation Metrics by Domain

- **ML:** Diagnosis accuracy (clean vs. messy images), Grad-CAM localization quality, crop recommendation accuracy
- **Security:** Adversarial attack success rate (before/after defense), prompt injection attempts blocked, API abuse/rate-limit test results, encryption coverage audit
- **Usability:** Task success rate, SUS score, NASA-TLX workload score, comprehension check accuracy
- **System:** Offline functionality coverage, sync integrity success rate, end-to-end latency (online vs. offline-then-sync)